# Strategies for Structuring Story Generation

**Angela Fan**
FAIR, Paris
LORIA, Nancy
angelafan@fb.com

**Mike Lewis**
FAIR, Seattle
mikelewis@fb.com

**Yann Dauphin**
Google AI*
ynd@google.com

arXiv:1902.01109v2 [cs.CL] 15 Jun 2019

## Abstract

Writers often rely on plans or sketches to write long stories, but most current language models generate word by word from left to right. We explore coarse-to-fine models for creating narrative texts of several hundred words, and introduce new models which decompose stories by abstracting over actions and entities. The model first generates the predicate-argument structure of the text, where different mentions of the same entity are marked with placeholder tokens. It then generates a surface realization of the predicate-argument structure, and finally replaces the entity placeholders with context-sensitive names and references. Human judges prefer the stories from our models to a wide range of previous approaches to hierarchical text generation. Extensive analysis shows that our methods can help improve the diversity and coherence of events and entities in generated stories.

## 1 Introduction

Stories exhibit structure at multiple levels. While existing language models can generate stories with good local coherence, they struggle to coalesce individual phrases into coherent plots or even maintain character consistency throughout a story. One reason for this failure is that classical language models generate the whole story at the word level, which makes it difficult to capture the high-level interactions between the plot points.

To address this, we investigate novel decompositions of the story generation process that break down the problem into a series of easier coarse-to-fine generation problems. These decompositions can offer three advantages:

- They allow more abstract representations to be generated first, where challenging long-range dependencies may be more apparent.

*Work done while at Facebook AI Research

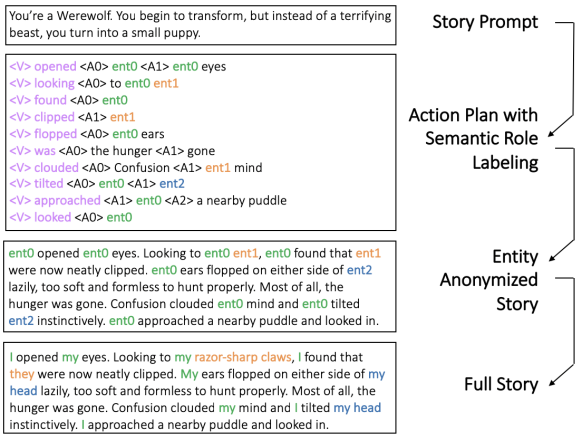


Figure 1: Proposed Model. Conditioned upon the prompt, we generate sequences of predicates and arguments. Then, a story is generated with placeholder entities such as *ent0*. Finally we replace the placeholders with specific references.

- They allow specialized modelling techniques for the different stages, which exploit the structure of the specific sub-problem.
- They are applicable to any textual dataset and require no manual labelling.

Several hierarchical models for story generation have recently been proposed (Xu et al., 2018; Yao et al., 2019), but it is not well understood which properties characterize a good decomposition. We therefore implement and evaluate several representative approaches based on keyword extraction, sentence compression, and summarization.

We build on this understanding to devise the proposed decomposition (Figure 1). Inspired by the classic model of Reiter and Dale (2000), our approach breaks down the generation process in three steps: modelling the action sequence, the story narrative, and lastly entities such as story characters. To model action sequences, we first generate the predicate-argument structure of the

story by generating a sequence of verbs and arguments. This representation is more structured than free text, making it easier for the model learn dependencies across events. To model entities, we initially generate a version of the story where different mentions of the same entity are replaced with placeholder tokens. Finally, we rewrite these tokens into different references for the entity, based on both its previous mentions and the global story context.

The models are trained on 300k stories from WRITINGPROMPTS (Fan et al., 2018), and we evaluate quality both in terms of human judgments and using automatic metrics. We find that our approach substantially improves story generation. Specifically, we show that generating the action sequence first makes the model less prone to generating generic events, leading to a much greater diversity of verbs. We also find that by using subword modelling for the entities, our model can produce novel names for locations and characters that are appropriate given the story context.

## 2 Model Overview

The crucial challenge of long story generation lies in maintaining coherence across a large number of generated sentences—in terms of both the logical flow of the story and the characters and entities. While there has been much recent progress in left-to-right text generation, particularly using self-attentive architectures (Dai et al., 2018; Liu et al., 2018), we find that models still struggle to maintain coherence to produce interesting stories on par with human writing. We therefore introduce strategies to decompose neural story generation into coarse-to-fine steps to make modelling high-level dependencies easier to learn.

### 2.1 Tractable Decompositions

In general, we can decompose the generation process by converting a story $x$ into a more abstract representation $z$. The negative log likelihood of the decomposed problem is given by

$$\mathcal{L} = -\log \sum_{z} p(x|z)p(z). \tag{1}$$

We can generate from this model by first sampling from $p(z)$ and then sampling from $p(x|z)$. However, the marginalization over $z$ is in general intractable, except in special cases where every $x$ can only be generated by a single $z$ (for example, if the transformation removed all occurrences of certain tokens). Instead, we minimize a variational upper bound of the loss by constructing a deterministic posterior $q(z|x) = 1_{z=z^*}$, where $z^*$ can be given by running semantic role labeller or coreference resolution system on $x$. Put together, we optimize the following loss:

$$z^* = \arg\max_{z} p(z|x) \tag{2}$$

$$\mathcal{L} \leq -\log p(x|z^*) - \log p(z^*) \tag{3}$$

This approach allows models $p(z^*)$ and $p(x|z^*)$ to be trained tractably and separately.

### 2.2 Model Architectures

We build upon the convolutional sequence-to-sequence architecture (Gehring et al., 2017). Deep convolutional networks are used as both the encoder and decoder. The networks are connected with an attention module (Bahdanau et al., 2015) that performs a weighted sum of the encoder output. The decoder uses a gated multi-head self-attention mechanism (Vaswani et al., 2017; Fan et al., 2018) to allow the model to refer to previously generated words and improve the ability to model long-range context.

## 3 Modelling Action Sequences

To decompose a story into a structured form that emphasizes logical sequences of actions, we use Semantic Role Labeling (SRL). SRL identifies *predicates* and *arguments* in sentences, and assigns each argument a *semantic role*. This representation abstracts over different ways of expressing the same semantic content. For example, *John ate the cake* and *the cake that John ate* would receive identical semantic representations.

Conditioned upon the prompt, we generate an SRL decomposition of the story by concatenating the predicates and arguments identified by a pre-trained model (He et al., 2017; Tan et al., 2018)[1] and separating sentences with delimiter tokens. We place the predicate verb first, followed by its arguments in canonical order. To focus on the main narrative, we retain only core arguments.

**Verb Attention Mechanism** SRL parses are more structured than free text, enabling more structured models. To encourage the model to

[1] for predicate identification, we use https://github.com/luheng/deep_srl, for SRL given predicates, we use https://github.com/XMUNLP/Tagger

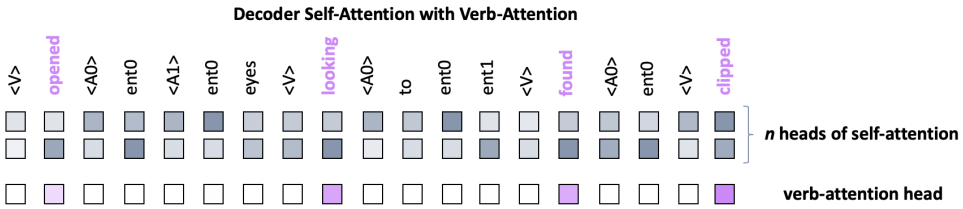


Figure 2: Verb-Attention. To improve the model's ability to condition upon past verbs, one head of the decoder's self-attention mechanism is specialized to only attend to previously generated verbs.

consider sequences of verbs, we designate one of the heads of the decoder's multihead self-attention to be a *verb-attention* head (see Figure 2). By masking the self-attention appropriately, this verb-attention head can only attend to previously generated verbs. When the text does not yet have a verb, the model attends to a zero vector. We show that focusing on verbs with a specific attention head generates a more diverse array of verbs and reduces repetition in generation.

## 4 Modelling Entities

The challenge of modelling characters throughout a story is twofold: first, entities such as character names are rare tokens, which make them hard to model for neural language models. Human stories often feature imaginative, novel character or location names. Second, maintaining the consistency of a specific set of characters is difficult, as the same entity may be referenced by many different strings throughout a story—for example *Bilbo Baggins*, *he*, and *the hobbit* may refer to the same entity. It is challenging for existing language models to track which words refer to which entity purely using a language modelling objective.

We address both problems by first generating a form of the story with different mentions of the same entity replaced by a placeholder token (e.g. *ent0*), similar to Hermann et al. (2015). We then use a sub-word seq2seq model trained to replace each mention with a reference, based on its context. The sub-word model is better equipped to model rare words and the placeholder tokens make maintaining consistency easier.

### 4.1 Generating Entity Anonymized Stories

We explore two approaches to identifying and clustering entities:

- **NER Entity Anonymization**: We use a named entity recognition (NER) model[2] to identify all people, organizations, and locations. We replace these spans with placeholder tokens (e.g. *ent0*). If any two entity mentions have an identical string, we replace them with the same placeholder. For example, all mentions of *Bilbo Baggins* will be abstracted to the same entity token, but *Bilbo* would be a separate abstract entity.
- **Coreference-based Entity Anonymization**: The above approach cannot detect different mentions of an entity that use different strings. Instead, we use the Coreference Resolution model from Lee et al. (2018)[3] to identify clusters of mentions. All spans in the same cluster are then replaced with the same entity placeholder string. Coreference models do not detect singleton mentions, so we also replace non-coreferent named entities with unique placeholders.

### 4.2 Generating Entity References in a Story

We train models to replace placeholder entity mentions with the correct surface form, for both NER-based and coreference-based entity anonymised stories. Both our models use a seq2seq architecture that generates an entity reference based on its placeholder and the story. To better model the specific challenges of entity generation, we also make use of a pointer mechanism and sub-word modelling.

**Pointer Mechanism** Generating multiple consistent mentions of rare entity names is challenging. To aid re-use of previous names for an entity, we augment the standard seq2seq decoder with a pointer-copy mechanism (Vinyals et al., 2015). To generate an entity reference, the decoder can either generate a new abstract entity token or choose to copy an already generated abstract entity token, which encourages the model to use consistent naming for the entities.

[2] https://spacy.io/api/entityrecognizer

[3] https://github.com/kentonl/e2e-coref